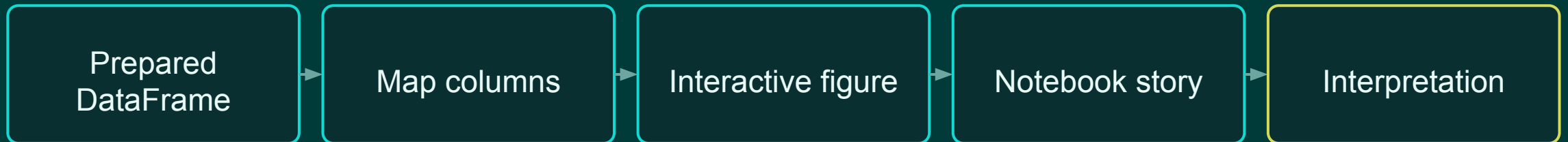


Interactive Visualization and Notebook Reporting

A one-block lesson with Plotly Express inside notebooks

Appendix D.2

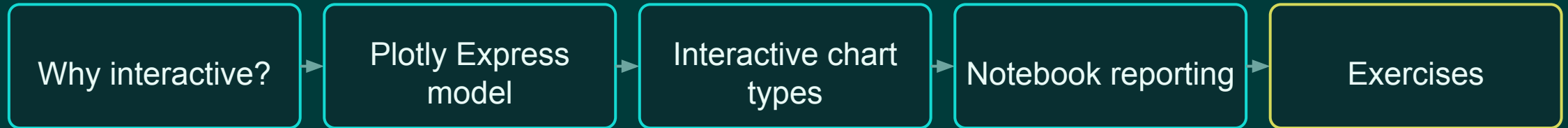
Use interaction only when it helps the reader inspect the answer.



Mental model: the notebook becomes a small analytical product, not just a place to run code.

Block Plan

30 minutes of instruction + 30 minutes of guided practice



Instruction

Static vs interactive, Plotly Express syntax, hover/zoom/legend controls, chart interpretation

Guided practice

Create bar, histogram, scatter, and weekly line charts from the equipment dataset

Applied output

A short notebook report with Markdown, code, interactive figures, and interpretation

Big idea: choose interaction because the question benefits from inspection, not because the library can do it.

Why Add Interactive Charts?

Static charts are often enough; interaction is for inspection and exploration



Hover over values

Inspect exact counts, record IDs, dates, or categories without adding clutter to the chart.



Zoom and pan

Explore a dense time window or a cluster of points without rebuilding the plot.



Filter with legends

Temporarily hide or show categories, such as individual source systems.

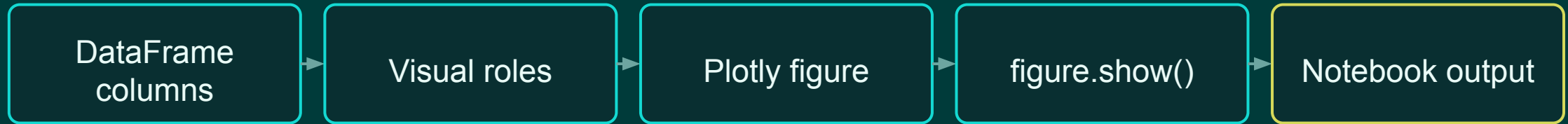
Ask first: would interaction help the viewer inspect the data?

If not, a static pandas / Matplotlib chart may be clearer.

interaction supports inspection

Plotly Express Mental Model

Map dataframe columns to visual roles



```
import plotly.express as px

figure = px.bar(
    source_summary,
    x="total_records",
    y="source",
    orientation="h",
)

figure.show()
```

Column mapping

x="total_records"
numeric horizontal position

y="source"
category position

orientation="h"
horizontal bars

Plotly Express is easiest when the values you want to plot are named columns.

Grouped Data Often Needs a Plot-Ready Shape

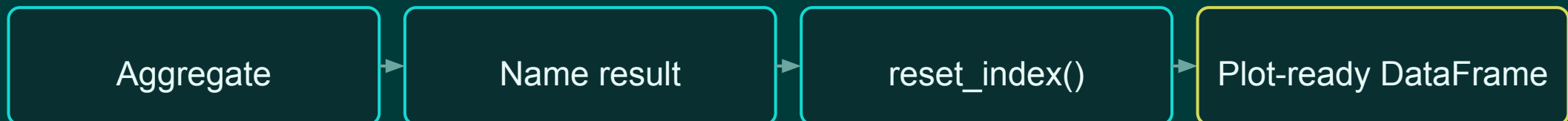
After aggregation, make the result easy for Plotly Express to read

```
source_totals = (
    records.groupby("source")["record_count"]
    .sum()
)

source_summary = (
    source_totals
    .rename("total_records")
    .reset_index()
)
```

Resulting dataframe

source	total_records
maintenance	734000
logistics	615000
finance	482000



This reinforces pandas: the chart is based on the prepared answer, not the raw dataframe.

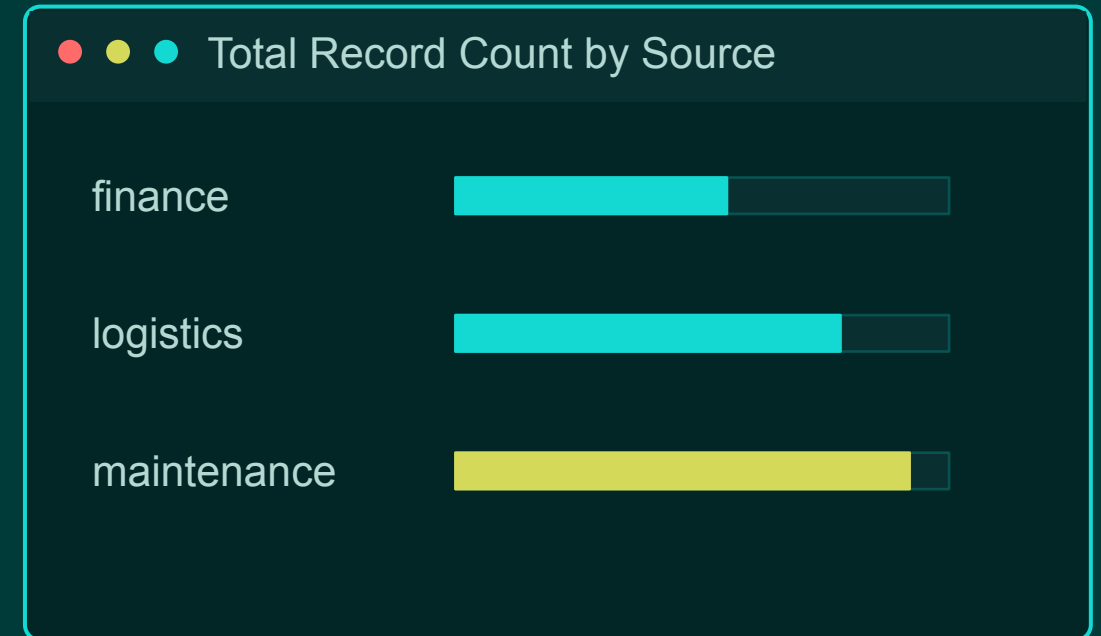
Interactive Bar Chart

Compare total record volume by source

```
source_summary = (
    records.groupby("source", as_index=False)
    .agg(total_records=("record_count",
    "sum"))
    .sort_values("total_records")
)

figure = px.bar(
    source_summary,
    x="total_records",
    y="source",
    orientation="h",
    title="Total Record Count by Source",
    labels={
        "total_records": "Total Records",
        "source": "Source System",
    },
)

figure.show()
```

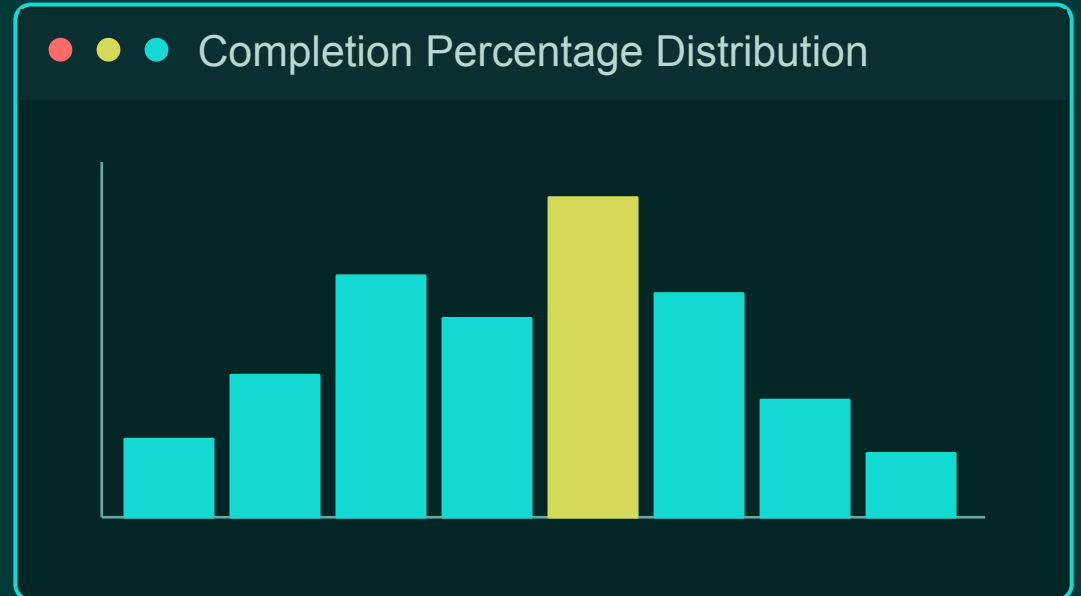


Question answered: which source contributed the greatest total record_count?

Interactive Histogram

Show the distribution of one numeric column

```
figure = px.histogram(  
    records,  
    x="completion",  
    nbins=10,  
    title=(  
        "Distribution of Completion "  
        "Percentages"  
    ),  
    labels={  
        "completion": "Completion  
Percentage",  
    },  
)  
figure.show()
```



nbins=5

broader view

nbins=15

more detail

Changing bins changes the level of detail, not the underlying data.

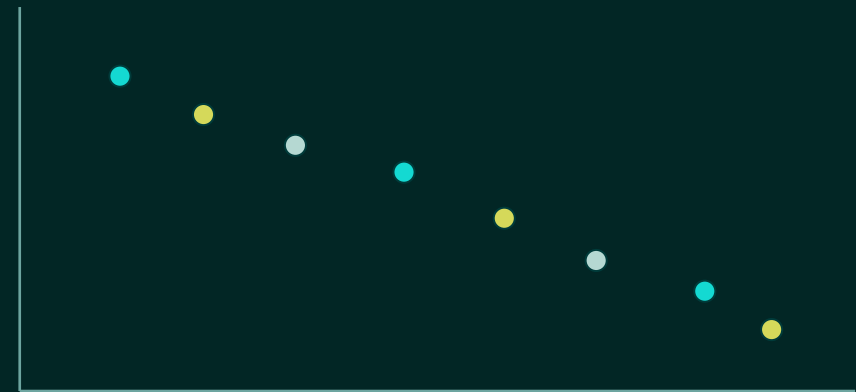
Interactive Scatter Plot

Inspect individual rows with hover data

```
figure = px.scatter(
    records,
    x="error_count",
    y="completion",
    color="source",
    hover_data=[
        "record_id",
        "status",
        "record_count",
        "processed_date",
    ],
    title="Errors Compared with Completion",
)

figure.show()
```

Errors Compared with Completion



Each point is one row. Hover details can identify unusual records without labeling every point.

Caution: a scatter plot can show association, but it does not prove causation.

Add Visual Encodings Carefully

More mappings can help, but they can also clutter the chart

```
figure = px.scatter(
    records,
    x="error_count",
    y="completion",
    color="source",
    size="record_count",
    hover_name="record_id",
    title=(
        "Completion, Errors, and "
        "Record Volume"
    ),
)

figure.show()
```

Visual roles

x and y
numeric relationship

color
category grouping

size
record volume

hover_name
record identity

Practical rule: add a visual encoding only when it helps answer the question.

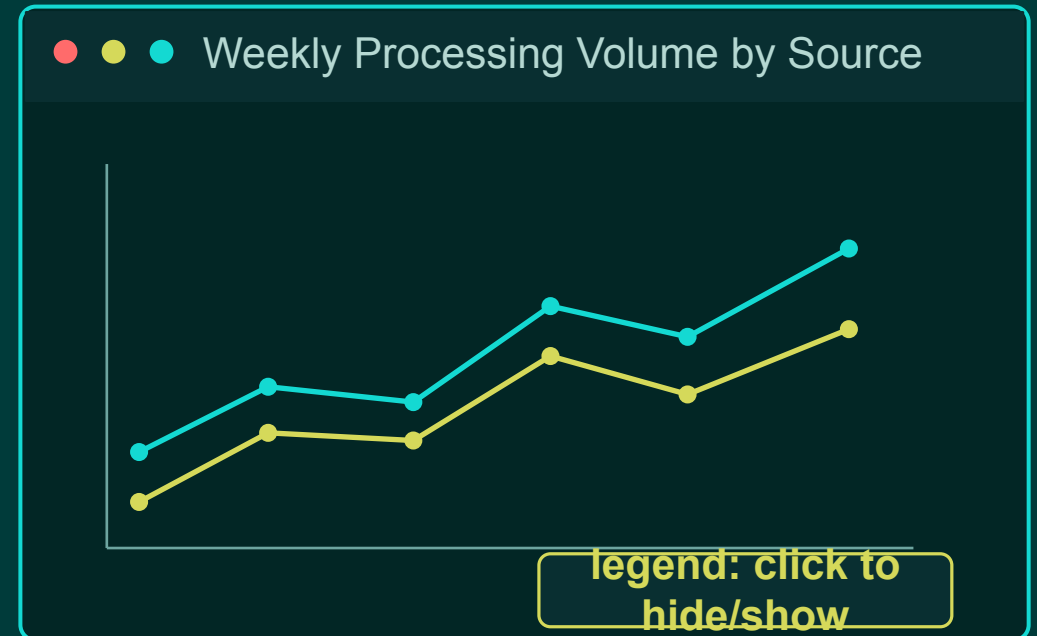
Interactive Time-Series Chart

Weekly totals by source support hover, zoom, and legend selection

```
weekly_by_source = (
    records.set_index("processed_date")
    .groupby("source")["record_count"]
    .resample("W")
    .sum()
    .rename("total_records")
    .reset_index()
)

figure = px.line(
    weekly_by_source,
    x="processed_date",
    y="total_records",
    color="source",
    markers=True,
    title="Weekly Processing Volume by Source",
)

figure.show()
```



Interaction adds value when the viewer may need to inspect a specific week or compare sources.

Titles and Labels Still Matter

Interactive charts should not require the reader to inspect the code

```
figure = px.bar(  
    source_summary,  
    x="source",  
    y="total_records",  
    title="Total Record Count by Source",  
    labels={  
        "source": "Source System",  
        "total_records": "Total Records",  
    },  
)  
  
figure.update_layout(  
    title_x=0.5,  
    legend_title_text="Source",  
)  
  
figure.show()
```

Weak title

Source

Better title

Total Record Count by
Source

Weak label

record_count

Better label

Total Records

The label should identify the metric, not
merely repeat the column name.

Static Versus Interactive Decision

Choose the rendering style based on audience and purpose

Static chart

Good for:

- simple reports
- printed documents
- image files
- fixed explanations
- stable classroom handouts

Strength: portable and predictable.

Interactive chart

Good for:

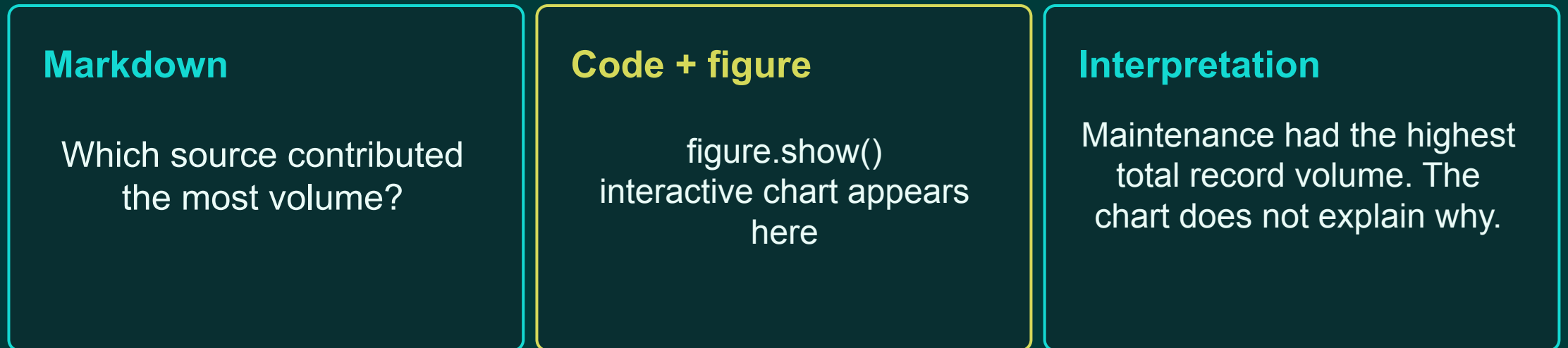
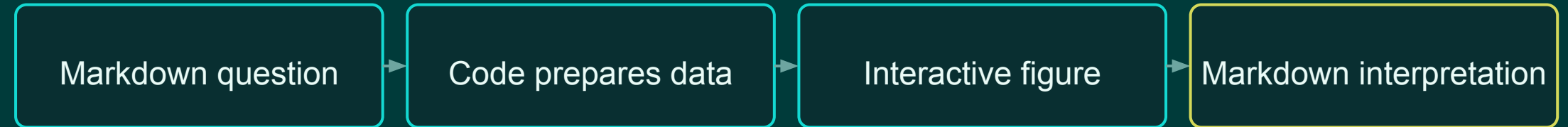
- exploration
- hover details
- zooming through time
- selecting categories
- inspecting records

Strength: deeper inspection.

Neither tool replaces the other. The question and delivery format decide.

Notebook as the Delivered Analytical Product

A reader should understand the analysis without sitting beside the programmer



A notebook is not merely a collection of cells. It is a communication document.

Every Chart Needs an Interpretation

Students should separate observations from unsupported conclusions

Supported observation

The weekly-volume chart shows maintenance had the highest volume during several middle weeks.

Why it is supported:
The chart directly displays weekly totals by source.

Unsupported conclusion

Maintenance was overloaded because the team had poor staffing.

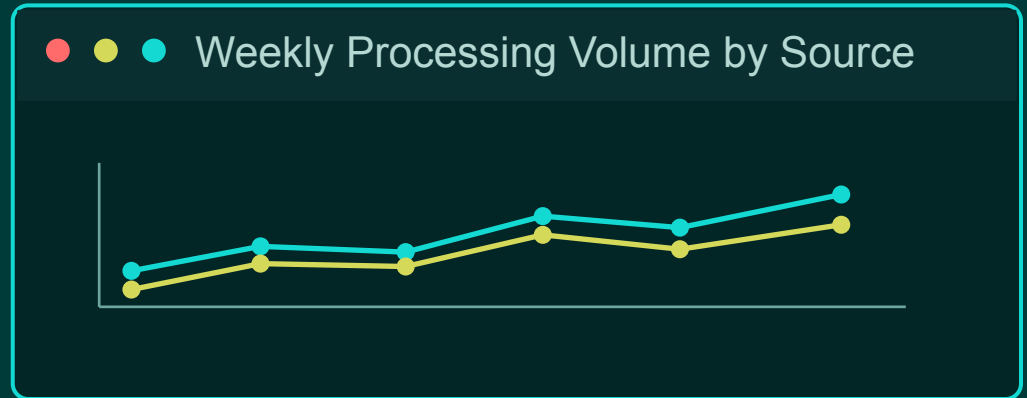
Why it is not supported:
The chart shows volume, not staffing or operational cause.

Interpretation should say what the visualization supports and what it does not answer.

Export an Interactive Chart

The notebook is primary, but HTML can share one interactive figure

```
figure.write_html(  
    "weekly_processing_volume.html"  
)
```



Plotly figure

Write HTML

Standalone interactive file

Static output: save a Matplotlib PNG for fixed reports or images.

Interactive output: save Plotly HTML when hover, zoom, or legend selection matters.

Common Errors to Address

Interactive charts can still mislead or confuse

Interaction as decoration

Do not make a chart interactive merely because the library allows it.

No interpretation

A chart without Markdown explanation is not a finished analysis.

Too many encodings

x, y, color, size, symbol, and hover can overwhelm the reader.

Unsupported causation

Scatter plots can reveal relationships, not prove causes.

Grouped index confusion

Plotly Express usually wants values in named dataframe columns.

Missing labels

Column names are often not good reader-facing labels.

Keep the analytical question in control of the chart, not the tool.

Guided Exercise Part 1: Setup and Summary Data

Create plot-ready data for interactive charts

```
import pandas as pd
import plotly.express as px

records = pd.read_csv(
    "block_42_equipment_activity.csv"
)

records["processed_date"] = pd.to_datetime(
    records["processed_date"]
)

source_summary = (
    records.groupby("source", as_index=False)
    .agg(total_records=("record_count", "sum"))
    .sort_values("total_records")
)
```

Tasks

1. Load the dataset
2. Parse processed_date
3. Inspect head() and info()
4. Group record_count by source
5. Make sure source and total_records are columns
6. Sort the result

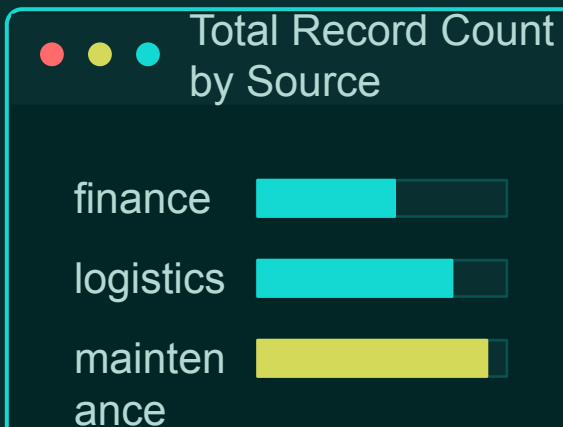
Time box: 10 minutes

Guided Exercise Part 2: Three Interactive Charts

Create charts that answer specific questions

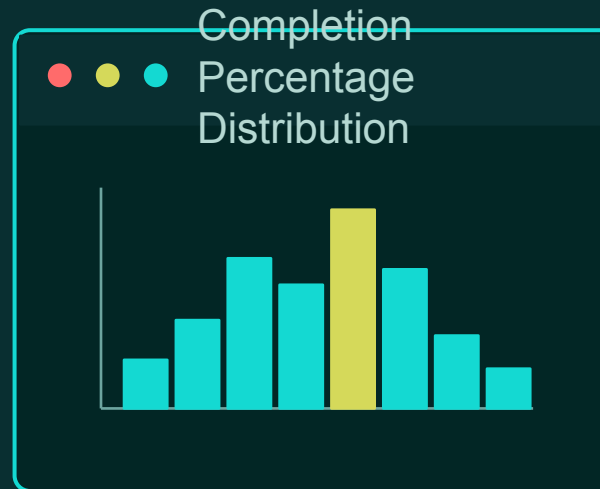
1. Source volume

Horizontal `px.bar()` with
total_records by source



2. Completion distribution

`px.histogram()`; compare
`nbins=5` and `nbins=15`



3. Record scatter

`px.scatter()`; `x=error_count`,
`y=completion`, `color=source`

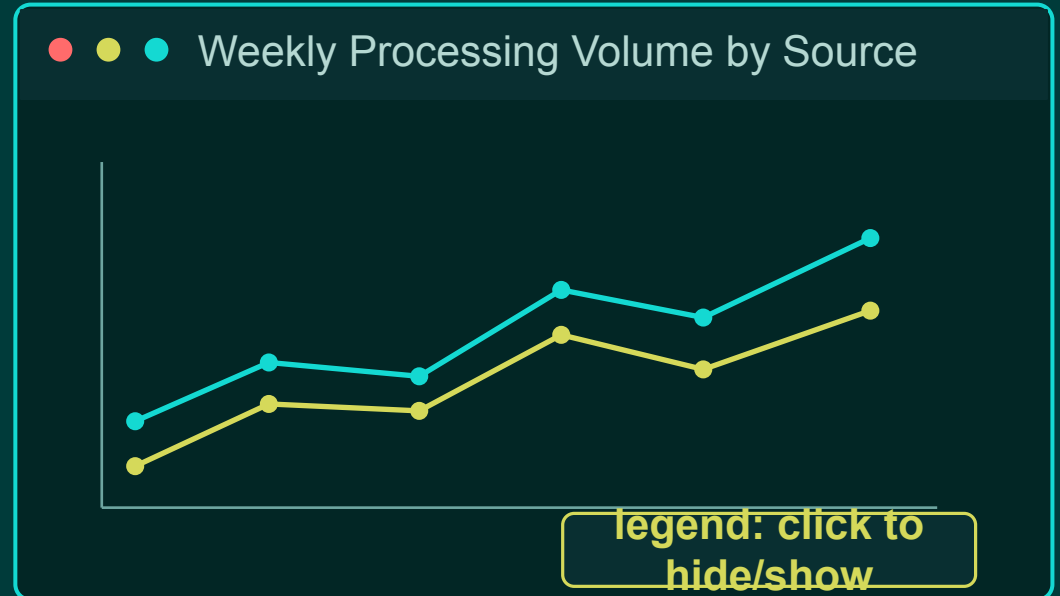


After each chart: write one observation and one limitation.

Guided Exercise Part 3: Weekly Trends and Explanation

Interactive time-series charts support exploration over dates

```
weekly_by_source = (  
    records.set_index("processed_date")  
    .groupby("source")["record_count"]  
    .resample("W")  
    .sum()  
    .rename("total_records")  
    .reset_index()  
)  
  
figure = px.line(  
    weekly_by_source,  
    x="processed_date",  
    y="total_records",  
    color="source",  
    markers=True,  
)  
  
figure.show()
```

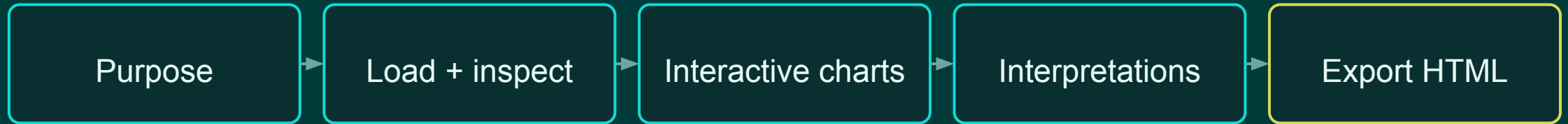


Markdown after the chart: one supported observation + one unanswered question.

Time box: 15 minutes

Applied Challenge: Interactive Notebook Mini-Report

Build a report a reader can understand without hearing the lecture



For every chart

Descriptive title • clear labels • useful hover details • Markdown observation • limitation or unanswered question

Knowledge Check and Wrap-Up

Students should be ready to use Plotly Express in notebook reports

When does interactivity improve a chart?

How is a static chart different from an interactive chart?

Why is `reset_index()` often useful before Plotly Express?

Why should every chart have Markdown interpretation?

What do `x=`, `y=`, `color=`, and `hover_data=` communicate?

What is the difference between an observation and an unsupported conclusion?

Final idea: interaction supports inspection, but the notebook still needs a question, calculation, chart, and explanation.